Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA [B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical]
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-27	Semester:	1st sem 3rd year
Name of the Student:	R. Prabhamsha	Roll Number:	A24126552171
Name of the Lab:	FS LAB	Timings:	9:40 to 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (5 Marks)	Programming Knowledge (5 Marks)	Results (5 Marks)	Viva-Voce (2Marks)			
1/7	1	3	4	4	2	13	10	
8/7	2	4	3	5	1	13	10	
15/7	3	4	4	5	1	14	10	
22/7	4	4	3	5	1	13	10	
29/7	5	4	4	5	1	14	10	
5/8	6	4	3	5	1	13	10	
12/8	7	4	3	5	1	13	10	
19/8	8	3	4	5	1	13	10	
2/9	9	3	4	5	1	13	10	
9/9	10	3	4	5	1	13	10	

Name of the Faculty:

Faculty Signature:

01/7/26

Introduction to HTML and CSS?

→ HTML (Hyper text markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of webpage using elements and tags such as headings, paragraphs links, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, background, spacing, boarder, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adapt different screen size and devices.

→ HTML and CSS work together to develop webpages, where HTML provides the structure and content while CSS provides the styling layout, visual presentation the combination of both forms the foundation of modern web development and enables the creation of organized attractive and user-friendly web interfaces.

Difference b/w MERN Stack and full Stack

→ MERN stack is a JavaScript based technology stack used for developing modern web applications MERN stands for MongoDB, Express.js, React.js, Node.js MongoDB is used for database management Express.js handles the backend framework and API's React.js is used to develop frontend user interface, and Node.js Provides the server side runtime environment.

→ Full Stack Development refers to the development of both the frontend and backend of a web application along with database integration A full stack developer works with technologies used for designing the user interface, implementing server-side logic managing database.

8/07/26

HTML components:

1. HTML Tags:- Define the structure and elements of a webpage
2. Headings - used to display titles and headings using <h1> to h6>
3. Paragraph - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <OL>, <UL>, <li>.
7) Tables - used to organize data in rows and columns
8. Forms - used to collect user input using elements such as text boxes, button and checkboxes
9. Div and Span - used to group and organize webpage content

CSS components

1) Selectors - Select HTML elements to apply styles
2. properties - Define the visual characteristics of elements.
3. Values - specify the setting of a CSS property
4. Colors - Define text, background, border colors
5. fonts - control font family, size, style and weight
6. backgrounds - set background colours and images
7. Borders - define the border style, width and radius.
8. Margin - controls the space outside an element
9. Padding - controls the space inside element
10. layout - Controls the arrangement of elements using flexbox, grid and positioning.

HTML Vs HTML5

HTML	HTML5
older version of HTML	latest major version of HTML
limited multimedia support	Support audio and video
Uses external plugins for multimedia	no external plugin needed
it has fewer semantic tags	Provides semantic tags
NO <Canvas> elements	supports <Canvas>
limited Web storage	supports local storage

CSS vs CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	provides advanced styling features
limited animations	supports animations and transitions
NO flexible layout system	Supports flexbox and Grid
limited visual effects	Supports Shadows, gradients, effects
NO media queries	Supports media queries
uses a single specification	Divide into different modules.

15/7/26

Design a static webpage using HTML and CSS

Pomodoro timer using HTML and CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            background : #f5f5f5;
            text-align : center;
            Padding: 50px;
        }
        .timer {
            width : 300px;
            margin: auto;
            Padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }
        h1 {
            Color : #e74c3c;
        }
        .time {
            font-size : 50px;
            margin: 20px
        }
        button {
            Padding: 10px 20px;
            margin: 5px;
            border: none;
            border-radius: 5px;
            background : #e74c3c;
            Color : white;
        }
    </style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer </h1>
        <div class = "time"> 25:00 </div>
        <button > Start </button>
        <button > Reset </button>
    </div>
</body>
</html>
```

22/07/2026

1) Design an HTML code using audio tag.

code:

```
<! DOCTYPE html>
<html>
<head>
    <title> Audio example </title>
</head>
<body>
    <h2> My favorite song </h2>
    <audio controls>
      <source src = "song.mp3" = "audio/mpeg">
      Your browser does not support the
      audio tag.
    </audio>
</body>
</html>
```

o/p: Display an audio player with play, pause and volume controls

Steps:

Create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls
Specify the audio file using <source> tag
Save and open the HTML file in a web browser
Play the audio using displayed Controll.

ii) Design a HTML code using video tag.

code:

```
<! DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My Video </h2>
    <video width = "400" controls>
      <source src = "video.mp4" type "video/mpu">
    </video>
</body>
</html>
```

o/p: Display a video player with play, pause, volume and full screen controls.

Steps:

Create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute
specify the video file using <source> tag
Save the file and open it in web browser
Play the video using displayed controls

write a program Javascript (arrays, functions)

code:

```
Let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray(arr){
    Console.log("Array Elements:");
    for (let i=0; i<arr.length; i++){
        Console.log(arr[i]);
    }
}
function findSum(arr){
    Let sum=0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

Experiment - 04

4) Develop dynamic webpage using java script

a) Program

```
<!DOCTYPE html>
<html>
<head>
  <title> Dynamic webpage</title>
  <style>
    body {
      text-align : center;
      font-family : Arial;
      margin-top : 100px;
    }
    button{
      Padding : 10px 20px;
      font-size : 16px;
      Cursor : pointer;
    }
  </style>
</head>
<body>
  <h1 id = "heading"> welcome to my webpage
  </h1>
  <p id = "message"> Click the button to
  change this webpage</p>
  <button on click = "changePage()" >click me
  </button>
```

```
<script>
function changePage(){
document.getElementById("heading").
    innerHTML = "Welcome to JavaScript!";
document.getElementById("message").
    innerHTML = "The Webpage has been
    changed dynamically";
document.body.style.backgroundColor
        = "lightblue";
    }
</script>
</body>
</html>
```

Output:

Welcome to my Webpage

Click the button to chang this webpage

Click Me

When the user click "Click Me", the javascript function changePage() is called, then the heading dynamically changes and background also changes

After clicking the button:

Welcome to JavaScript!

The Webpage has been changed dynamically

Click Me

Experiment-05

5) implement modules in nodeJS

In Node.JS, You can implement modules using two distinct systems: CommonJS(CJS) which is traditional default, and ES Modules (ESM) the modern JavaScript Standard.

1) The common JS Method (default):

CommonJS uses module.exports to share code and require() to import it this system loads modules synchronously

a) Create the module (math.js):

```
Const add = (a,b) => a+b;
Const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract
};
```

b) Use the module (app.js):

```
const math = require ('./math');
console.log (math.add (10,5));
Console.log (math.subtract (10,5));
```

Output: 15
5

2) The ES modules Method (modern Standard)

ES modules use Export to share code and import use it To make Node.JS recognize ES module, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a) Create the module (Calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
Const calculatorName = "SuperCalc";
export default calculatorName;
```

b) use the module (main.js)

```
import calculatorName, {multiply, divide} from './calculator.js';
Console.log (calculatorName);
Console.log (multiply (4,2));
Console.log (divide (4,2));
```

Output: SuperCalc
8
2

3) Using Built-in core modules:

Node.js also comes equipped with dozens of built in core modules like fs for file systems or path for file routing that dont need installation. We will load them by modules names:

```
// CommonJS style
const fs = require('fs');

// ES modules style
import fs from 'fs';
```

Here are some of most commonly used ones:

1. fs = file system operations
2. http - HTTP server and client
3. path - file path utilities
4. os - operating system utilities
5. events - Event handling
6. until - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. URL - URL parsing
10. querystring - URL query string handling

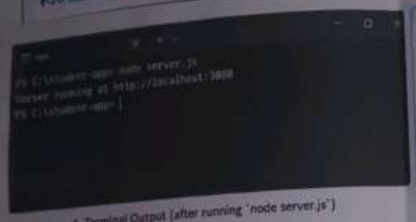
Example: simple HTTP server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
        'text/html'});
    res.end('Hello world');
}).listen(8080);
```

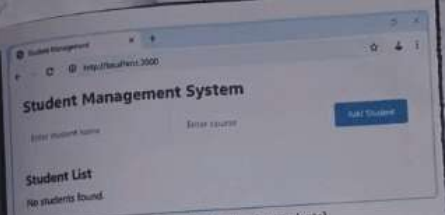
Output: Hello world!

Experiment-06

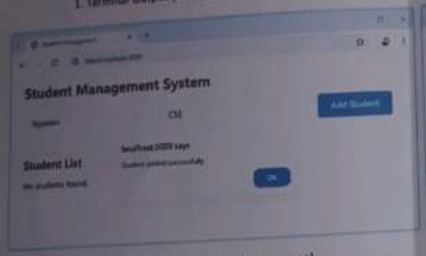
6) Develop mini application using express and Node JS



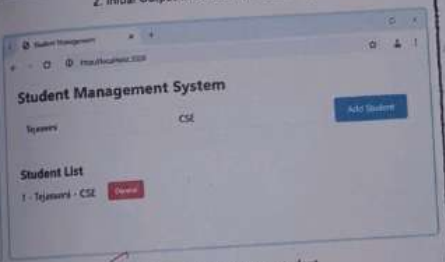
1. Terminal Output (after running 'node server.js')



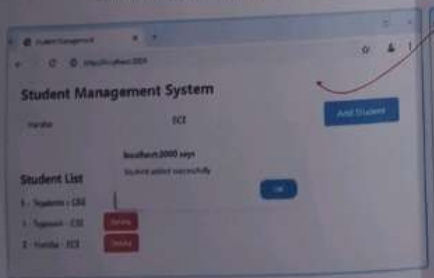
2. Initial Output in Browser (no students)



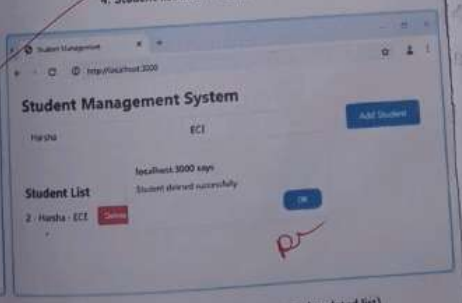
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Student Management System

1. Add a Student
2. View all students
3. delete a Student
4. Uses Express Js routes
5. Simple HTML interface

Project Structure:

```
student-app/
├── server.js
└── public
    └── index.html
```

server.js

```
Const express = require ("express");
Const app = express ();
Const port = 3000;
Let students = [];
app.use (express.json());
app.use (express.static ("public"));
app.get ("/students", (req, res)=>{
    res.json (students);
});
// Add
app.post ("/students", (req, res)=>{
    Const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push (student);
    res.json ({message: "student added succesfully",
        student: student});
});
// Delete
app.delete ("/students/:id", (req, res)=>{
    Const id = parseInt (req.params.id);
    Students = students.filter (student =>
        student.id !== id);
    res.json ({
        message: "student deleted succesfully"});
});
// Start Server
app.listen (PORT, ()=>{
    console.log (`server running at http://localhost
        ${PORT}`);
});
```

Public /index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
```

```
<h1> Student Mangement System </h1>
<input type = "text" id= "name"
placeholder = "Enter student name" >
<input type = "text" id = "course"
placeholder = "Enter course" >
<button onClick= "addStudent()" >
  Add Student </button>
<h2> Student list </h2>
<ul id = "studentlist" > </ul>
<script>
function addStudent(){
let name = document.getElementById("name").
            value;
Let course = document.getElementById("course").
            value;
fetch("/students", {
    method: "POST",
    headers: {
        "Content-Type": 'application/json' },
body: JSON.stringify({
    name: name,
    course: course
})
})
.then(response => response.json())
.then(data => {
    alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudent();
});
}
function displayStudents(){
fetch("/students").then(response => response.json())
.then(students => {
  let list = document.getElementById("studentlist")
  list.innerHTML = "";
students.forEach(student => { let li =
document.createElement("li")
    li.innerHTML = student.id + "_" +
                   student.name + "_" +
                   student.course + "<button
onClick = "deleteStudent(" + student.id + ")">Delete
</button>";
list.appendChild(li);
});
});
}
```

```
function deleteStudent(id){
  fetch(`/students/${id}`,{method:'DELETE'})
  .then(response=>response.json())
  .then(data=>{
    alert(data.message);
    displayStudent();
  });
}
displayStudents();
</script>
</body>
</html>
```

Output:

Student Management System.

Enter student name

Enter course

Add Student

Student list

1. Raju - CSM delete
2. Sita - CSE delete

- The Express.js server successfully started at http://localhost:3000
The browser displays the student management system with fields

Experiment - 07

7) create a functional component and class component using react

Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a web page.

Functional Component:

A functional component is a javascript function that returns JSX to display to user interface. It uses React hooks such as useState to manage state and and update the webpage dynamically

Program:

```
import React, {useState} from
'react';
function StudentFunction(){
  const[name,setName]=useState("Rajuram");
  const[course,setCourse]=useState("CSM");
  Const [count,setCount]=useState(0);
  const changeName=()=>{
    setName("Teju");
  };
  Const changeCourse=()=>{
    setCourse("CSD");
  };
```

```
Const incrementCount = () => {
    setCount(count + 1);
};
return (
  <div style={{
      border: "2px solid blue",
      padding: "20px",
      margin: "20 px",
      borderRadius: "10px",
      textAlign: "center"
  }}>
  <h1> Student Details </h1>
  <h2> Functional Component </h2>
  <p><b> Name: </b>{name}</p>
  <p><b> course: </b>{course}</p>
  <p><b> Button Clicks: </b>{count}</p>
  <button onClick={changeName}>
      Change Name </button>
  <button onClick={changeCourse}>
      Course </button>
  <button onClick={incrementCount}>
      </button>
  </div>
);
```

output:

When the user clicks the buttons, the name, course and count change dynamically without reloading the webpage

Class component:

A class component is an ES6 javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

Program:

```
import React, {Component} from 'react';
class Student Class extends Component {
    constructor (props){
        super (props);
        this.state ={
            name : "Sowjith",
            Course : "Computer Science",
            count: 0
        }
    }
    ChangeName = () => {
        this.setState({
            name: "Teja"
        });
    };
    ChangeCourse =() =>{
        this.setState({
            Course : "CSM"
        });
    };
    incrementCount=() =>{
        this.setState((Prevstate) =>{
            Count : prev State.Count +1
        }));
    };
render(){
  return (
    <div style ={{
        border = "2px" solid green",
        padding :"20 px",
        margin :"20px",
        borderRadius: "10px",
        text Align :"center"
        }}>
        <h1> Student Details </h1>
        <h2> Class Component </h2>
        <p> <b>Name: </b> {this.state.name}</p>
        <p><b>course: </b> {this.State.Course}</p>
        <p><b> Button Clicks</b> {this.state.count}</p>
    <button onClick = {this.ChangeName}>
            ChangeName </button>
    <button onclick ={this.ChangeCourse}>
            Change Course </button>
    <button onClick ={this.incrementCount}
            > increment
        </button>
    </div>
```

Output:

Experiment-08

React App

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step 2: Create db.json

Create a file named db.json and add the following student data

```
{
  "students": [
    {
      "id": "1",
      "name": "Raju",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "Ramu",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3",
      "name": "Ravi",
      "course": "IT",
      "marks": 88
    }
```

Step-3: Start the JSON Server

For the current JSON Server version, execute:

npx json server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods

a. Get Method The GET Method is used to retrieve student records from JSON server

Code:

```
fetch ('http://localhost:3000/students')
.then (response => response.json())
.then (data => {
        console.log(data);
    })
.catch (error => {
        console.log(error);
    });
```

output:

B. POST Method: The POST method is used to insert a new Student record into the server

Code:
```
fetch("http://localhost:3000/students",{
    method:'POST',
    headers:{
    'Content-Type':'application/json'
    },
    body:JSON.stringify({
        name:'Raju',
        Course:'CSE',
        marks:92
    })
})
.then(response=>response.json())
.then(data=>{
    console.log(data);
});
```

```
.catch(error=>{
    console.log(error);
});
```

output:

C. PUT Method: The PUT method is used to Replace an existing Student record with updated data

Code:
```
fetch('http://localhost:3000/students/1',{
    method:'PUT',
    headers:{
    'Content-Type':'application/json'
    },
    body:JSON.stringify({
        name:'Raju',
        Course:'AIML',
        marks:95
    })
})
.then(response=>response.json())
```

output:

- After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining students will be

output:

Thus, HTTP methods GET, POST, PUT, PATCH, DELETE were sucessfully implemented using JSON Server. The CRUD operations were performed on students records, the data was retrieved inserted, updated and deleted using HTTP requests.

Experiment - 9

9) Develop CRUD Operations using MongoDB

Aim: To develop a web application using Node.js, Express.js and MongoDB to perform CRUD Operations (Create, read, update, delete) on students records

Theory: MongoDB is a NOSQL database that stores data in flexible, JSON like documents, CRUD operations are used to manage data in a database.

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and Project setup:

```
mkdir MongoDB_CRUD
cd MONGODB_CRUD
npm init -y
npm install express mongoose
```

Program: Server.js

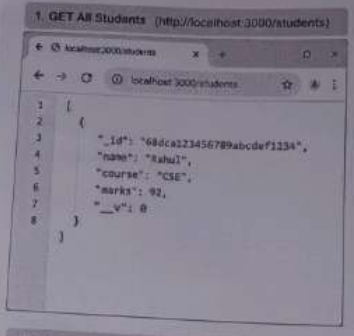
```
Const expres. require ('expres')
const mangoose. require ('mongoose');
Const app = expres ();
Const PORT = 3000;
app. use (expres. json());
mogoose connect ('mongodb://127.0.0.1:27017/
                  college)
.then(() => console.log ("MongoDB Connected"))
.catch(err => console. log(err));
//Create Student
Const Student Schema = new
mongoose Schema ({
    name : {
        tape: String,
        required: true
    },
    course : {
        type : String,
        required: true
    },
    marks:{
        type : NUmber
        required! true
    }
}
});
//Create Student model
const Student :
    mongoose. model ('Student', StudentSchema)
//CREATE ; Add
app Post ('/students', asyn(req,res)=>{
    try{
        const student = new Student (req.body);
        Const savedStudent = await Student
                                save();
        res.status(201).json(savedStudent)
    } catch(error){
        res. status (400).json({message
            error.message});
    }
});
// Read : Get all Students
app.get ('/students', asyn(req,res) =>{
    try{
        Const Students = await
```

```
Student.find();
  res.status(500).json({message: error message});
  }
});
//READ: Get student by ID
app.get('/student/:id', async(req,res)=>{
  try{
    const student = await
    Student.findById(req.params.id)
    if(!student){
      return res.status(404).json({
        message: "student not found"
      });
    }
    res.status(200).json(student)
  }catch(error){
    res.status(400).json({message:
      error.message});
  }
});
//UPDATE
app.put('/student/:id',async((req,res)=>{
  try{
    const student = await
    Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {new: true, runValidators: true}
    );
  if(!student){
    return res.status(404).json({
      message: "student not found"
    });
  }
  res.status(200).json(student);
  }catch(error){
  res.status(400).json({message:
    error.message});
  }
});
//DELETE
app.delete('/student/:id', async(req,res)=>{
  try{
  const student = await student.
    findByIdAndDelete(
```

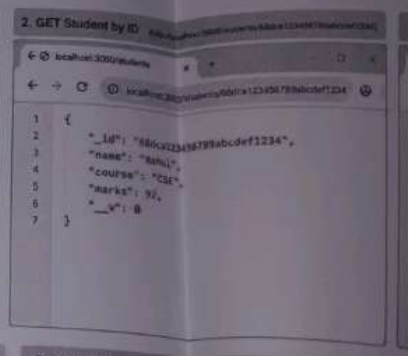
```
req.params.id
});
if(!student){
  return res.status(404).json({
    message: "Student not found"
  });
}
res.status(200).json({
  message: "Student deleted
            Succesfully"
});
} catch(error){
  res.status(400).json({message:
    error.message});
}
});
```

Output:

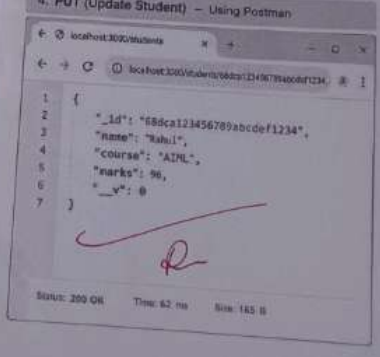
1. GET All Students (http://localhost:3000/students)



2. GET Student by ID



4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman

